

Kompleksowy Poradnik: Integracja Circle Paymaster z TipJar

****Data utworzenia:** 11 maja 2025**

****Wersja:** 1.0**

****Cel Poradnika:**** Umożliwienie fanom korzystającym z własnych, zewnętrznych portfeli EOA (np. MetaMask) płacenia napiwków w USDC na platformie TipJar, gdzie opłaty transakcyjne (gaz) są również pokrywane w USDC za pomocą usługi Circle Paymaster. Eliminuje to potrzebę posiadania przez fana natywnych tokenów sieci (jak ETH, MATIC) do uiszczania opłat za gaz.

****Kontekst Integracji w TipJar:****

*** **Circle Gas Station (dla transakcji wewnętrznych/twórców):**** W podstawowym modelu TipJar, dla transakcji inicjowanych z portfeli kontrolowanych przez dewelopera (DCW) należących do twórców (np. wypłaty) lub ewentualnych wewnętrznych portfeli TipJar, platforma TipJar będzie sponsorować opłaty za gaz za pomocą usługi ****Circle Gas Station****. Rozliczenie z Circle odbywa się w walucie fiat (np. kartą kredytową dewelopera).

*** **Circle Paymaster (dla fanów z własnymi EOA):**** Integracja z Circle Paymaster będzie stanowiła ****dodatkową, opcjonalną metodę płatności dla fanów****, którzy preferują korzystanie ze swoich własnych, zewnętrznych portfeli EOA. W tym scenariuszu to fan pokrywa koszt gazu, ale płaci za niego w USDC (plus ewentualna prowizja Circle Paymaster), a nie w natywnym tokenie danej sieci blockchain.

****Kiedy i Gdzie Integrować Circle Paymaster?***

*** **MVP vs. Dalszy Rozwój:****

*** **Rekomendacja dla MVP:**** Biorąc pod uwagę, że problem opłat za gaz jest znaczącą barierą dla użytkowników Web3, a Circle Paymaster (szczególnie w połączeniu z nadchodzącym EIP-7702 dla EOA) efektywnie go rozwiązuje, ****warto rozważyć włączenie integracji z Circle Paymaster już na etapie MVP****. Zaoferuje to fanom większą elastyczność i obniży próg wejścia. Dodatkową zachętą jest promocja Circle, znosząca 10% opłatę Paymastera do 1 lipca 2025 roku.

*** Jeśli MVP ma być skrajnie uproszczone, można początkowo skupić się na płatnościach kartą (z konwersją fiat-USDC w tle przez TipJar) i/lub na transakcjach, gdzie fani posiadają tokeny natywne. Paymaster byłby wtedy dodany jako jedna z pierwszych funkcji po MVP.**

*** **Miejsce Integracji w Architekturze TipJar:****

*** **Frontend Aplikacji TipJar (Web/Mobile):**** Główna logika integracji z Circle Paymaster będzie znajdować się po stronie klienta. Obejmuje to:

- * Interakcję z portfelem fana (np. MetaMask).
- * Konstruowanie `UserOperation` (zgodnie ze standardem ERC-4337).
- * Podpisywanie pozwolenia `Permit` (EIP-2612) dla kontraktu USDC.
- * Wysyłanie `UserOperation` do wybranego Bundlera z danymi Circle Paymaster.

*** **Backend Aplikacji TipJar (NestJS):**** Może pełnić rolę pomocniczą:

* Dostarczanie aktualnych adresów kontraktów Circle Paymaster i USDC dla różnych sieci.

* Potencjalnie pomoc w estymacji opłat za gaz (choć często robią to biblioteki klienckie lub Bundlery).

* Odbieranie i weryfikowanie statusu transakcji (np. poprzez nasłuchiwanie na zdarzenia kontraktu Paymaster lub monitorowanie portfela twórcy).

* Backend ****nie zarządza**** kluczami API dla Circle Paymaster, ponieważ jest to usługa "permissionless" (bez zezwoleń).

****Szczegółowe Kroki Integracji Circle Paymaster (ERC-4337 / EIP-7702)****

Poniższy przewodnik opiera się na koncepcji ERC-4337 (Account Abstraction) i wykorzystaniu Paymasterów. Dzięki EIP-7702, standardowe portfele EOA będą mogły tymczasowo działać jak Smart Contract Accounts (SCA) na czas jednej transakcji, co upraszcza proces dla użytkownika końcowego. Należy celować w ****Circle Paymaster v0.8**** ze względu na szersze wsparcie sieci i EntryPoint v0.8.

****Założenia:****

* Fan inicjuje napiwek USDC z własnego portfela EOA (np. MetaMask).

* Fan posiada wystarczającą ilość USDC na pokrycie kwoty napiwku oraz szacunkowego kosztu gazu w USDC (wraz z ewentualną opłatą dla Paymastera).

* Frontend TipJar korzysta z biblioteki JavaScript/TypeScript takiej jak `viem` do interakcji z portfelem fana i blockchainem.

****Faza 1: Ustawienia i Inicjalizacja (Frontend Aplikacji TipJar)****

1. **Instalacja Niezbędnych Zależności:**

Do projektu frontendowego TipJar należy dodać odpowiednie biblioteki:

```
```bash
npm install --save viem dotenv
lub
yarn add viem dotenv
```
```

* `viem`: Nowoczesna biblioteka do interakcji z Ethereum i sieciami EVM-kompatybilnymi. Służy do tworzenia klientów, interakcji z kontraktami, podpisywania wiadomości/transakcji, obsługi ERC-4337 (UserOperations).

* `dotenv`: Do zarządzania zmiennymi środowiskowymi w środowisku deweloperskim.

* Może być potrzebna dodatkowa biblioteka do obsługi Account Abstraction, jeśli `viem` nie pokrywa wszystkich potrzeb w prosty sposób (np. starsze wersje quickstartów Circle używały `permissionless` lub `@alchemy/aa-core`). Nowsze quickstarty Circle dla Paymaster v0.8 z EIP-7702 wykorzystują `viem/account-abstraction`.

2. **Konfiguracja Zmiennych Środowiskowych (Plik `.env` na Frontendzie):**

Należy zdefiniować adresy kontraktów i inne stałe dla każdej wspieranej przez TipJar sieci. Przykładowo dla Arbitrum Sepolia (testnet):

```
```text
```

NEXT\_PUBLIC\_ARBITRUM\_SEPOLIA\_RPC\_URL=[https://sepolia-rollup.arbitrum.io/rpc](https://sepolia-rollup.arbitrum.io/rpc)

NEXT\_PUBLIC\_ARBITRUM\_SEPOLIA\_USDC\_ADDRESS=0x75faf114eafb1BDbe2F0316DF893fd58CE46AA4d

NEXT\_PUBLIC\_ARBITRUM\_SEPOLIA\_PAYMASTER\_V08\_ADDRESS=0x3BA9A96eE3eF3A69E2B18886AcF52027EFF8966

NEXT\_PUBLIC\_ARBITRUM\_SEPOLIA\_ENTRYPOINT\_V08\_ADDRESS=  
<ADRES\_ENTRYPOINT\_V0.8\_NA\_TEJ\_SIECI>

NEXT\_PUBLIC\_PIMLICO\_BUNDLER\_API\_KEY=YOUR\_PIMLICO\_API\_KEY\_IF\_NEEDED

NEXT\_PUBLIC\_PIMLICO\_BUNDLER\_URL\_ARBITRUM\_SEPOLIA=[https://api.pimlico.io/v2/arbitrum-sepolia/rpc](https://api.pimlico.io/v2/arbitrum-sepolia/rpc)

...

\* Adresy Paymastera i USDC dla różnych sieci znajdują się w dokumentacji Circle ("Paymaster Addresses and Events").

\* Adresy EntryPoint dla ERC-4337 v0.8 (lub v0.7) są standardowe dla danych sieci.

\* URL Bundlera: Należy wybrać publicznego dostawcę Bundlera (np. Pimlico, Alchemy, Stackup) i użyć jego RPC URL. Niektórzy mogą wymagać klucza API.

### 3. \*\*Inicjalizacja Klientów `viem` i Konta Użytkownika (w Logice Frontendu):\*\*

W momencie, gdy fan decyduje się na wysłanie napiwku ze swojego zewnętrznego portfela EOA:

\* \*\*Połączenie z Portfelem Fana:\*\* Aplikacja TipJar musi poprosić fana o połączenie jego portfela (np. MetaMask). Po pomyślnym połączeniu, uzyskuje dostęp do adresu fana (`fanEOA\_Address`) i możliwości wysyłania żądań podpisania.

\* \*\*Utworzenie Klientów `viem`:\*\*

```
``typescript
import { createPublicClient, createWalletClient, http, custom } from 'viem';
import { arbitrumSepolia } from 'viem/chains'; // Przykładowa, aktywna sieć
import { toSimple7702SmartAccount, ENTRYPOINT_ADDRESS_V07,
ENTRYPOINT_ADDRESS_V06 } from 'viem/account-abstraction'; // Dla EIP-7702

// Dynamiczne ustawienie na podstawie wyboru użytkownika lub kontekstu twórcy
const currentChain = arbitrumSepolia;
const usdcContractAddress =
process.env.NEXT_PUBLIC_ARBITRUM_SEPOLIA_USDC_ADDRESS as `0x${string}`;
const paymasterAddress =
process.env.NEXT_PUBLIC_ARBITRUM_SEPOLIA_PAYMASTER_V08_ADDRESS as
`0x${string}`;
// ENTRYPOINT_ADDRESS_V07 lub V06 może być potrzebny zależnie od
Paymastera Circle (dokumentacja wskazuje V0.7 i V0.8)
// Dla Paymaster v0.8 użyjmy odpowiedniego adresu EntryPoint v0.7/v0.8
const entryPointAddress = ENTRYPOINT_ADDRESS_V07; // Sprawdź dokumentację
Circle dla Paymaster v0.8
```

```

// Klient publiczny do odczytu danych z blockchajna
const publicClient = createPublicClient({
 chain: currentChain,
 transport: http(process.env.NEXT_PUBLIC_ARBITRUM_SEPOLIA_RPC_URL), // Lub
RPC URL od Alchemy/Infura
});

// Klient portfela do interakcji z portfelem fana (np. MetaMask)
const walletClient = createWalletClient({
 chain: currentChain,
 transport: custom(window.ethereum), // Zakładając, że window.ethereum jest
dostępne
});

const [fanEOA_Address] = await walletClient.getAddresses();
const fanEOA_Account = { address: fanEOA_Address, type: 'json-rpc' as const }; // Typ
dla viem

// Utworzenie "tymczasowego" Smart Account dla EOA dzięki EIP-7702
// To jest kluczowy krok z quickstartu Circle dla Paymaster v0.8
const eip7702SmartAccount = await toSimple7702SmartAccount({
 publicClient,
 signer: walletClient, // viem automatycznie użyje podłączonego konta
 owner: fanEOA_Account, // Przekazanie obiektu konta z viem
 entryPoint: entryPointAddress,
 // factoryAddress:
<ADRES_FABRYKI_DLA_SIMPLE_7702_ACCOUNT_JEŚLI_WYMAGANY>
 // Inne opcje konfiguracyjne, jeśli potrzebne
});
...

```

---

**\*\*Faza 2: Sprawdzenie Salda USDC Fana i Konfiguracja Pozwolenia `Permit` (Frontend TipJar)\*\***

Zanim fan wyśle napiwek, aplikacja frontendowa TipJar musi wykonać dwie kluczowe operacje: sprawdzić, czy fan ma wystarczająco USDC na pokrycie napiwku i opłat za gaz, oraz uzyskać od fana kryptograficzne pozwolenie (`Permit` zgodne z EIP-2612) dla kontraktu Circle Paymaster na pobranie tych USDC z jego konta.

#### 1. **\*\*Sprawdzenie Salda USDC Fana:\*\***

\* Używając `publicClient` z `viem` oraz ABI kontraktu ERC20 (które `viem` często ma wbudowane), odczytaj saldo USDC na adresie `eip7702SmartAccount.address` (lub bezpośrednio `fanEOA\_Address`, jeśli Paymaster operuje na saldzie EOA przed "transformacją" w SCA). Quickstart Circle dla Paymaster v0.8 wskazuje na sprawdzanie salda `account.address` gdzie `account` to `toSimple7702SmartAccount`.

```typescript

```

import { erc20Abi, formatUnits, parseUnits } from 'viem';

const usdcContract = getContract({
  address: usdcContractAddress,
  abi: erc20Abi,
  client: publicClient, // lub walletClient, jeśli operacja wymaga połączonego konta
});

const fanAddressForBalance = eip7702SmartAccount.address; // Adres, którego saldo
sprawdzamy
const usdcBalanceBigInt = await usdcContract.read.balanceOf([fanAddressForBalance]);
const usdcDecimals = await usdcContract.read.decimals(); // Zwykle 6 dla USDC
const usdcBalanceFormatted = formatUnits(usdcBalanceBigInt, usdcDecimals);

const tipAmount = parseUnits("5", usdcDecimals); // Napiwek 5 USDC
const estimatedGasInUSDC = parseUnits("0.1", usdcDecimals); // Przykładowa estymacja
kosztu gazu w USDC

if (usdcBalanceBigInt < (tipAmount + estimatedGasInUSDC)) {
  console.log(
    `Niewystarczające saldo USDC. Potrzebujesz około ${formatUnits(tipAmount +
estimatedGasInUSDC, usdcDecimals)} USDC, a masz ${usdcBalanceFormatted} USDC na
adresie ${fanAddressForBalance} na sieci ${publicClient.chain.name}. Użyj
https://faucet.circle.com, aby doładować testowe USDC, a następnie spróbuj ponownie.`
  );
  // Poinformuj użytkownika w UI
  // process.exit(); // W realnej aplikacji, nie wychodź, tylko pokaż błąd
  return;
}
...

```

2. ****Implementacja Podpisywania `Permit` (EIP-2612) dla USDC:****

* Circle Paymaster, aby móc pobrać USDC z konta fana na pokrycie opłat za gaz, potrzebuje na to jego zgody. Zamiast tradycyjnej transakcji `approve` (która sama kosztuje gaz), wykorzystuje się mechanizm podpisanych pozwoleń `Permit` (standard EIP-2612).

* Kod do generowania struktury danych `Permit` i jej podpisywania przez fana jest przedstawiony w quickstarcie Circle (plik `permit.js`). Należy zaadaptować tę logikę w kodzie frontendu TipJar.

****Kluczowe:**** Wartość `deadline` w wiadomości `Permit` musi być ustawiona na `maxUint256` (maksymalna wartość typu `uint256`), ponieważ smart kontrakt Paymastera, działając w ramach ograniczeń ERC-4337, nie ma dostępu do `block.timestamp` podczas walidacji.

```

````typescript
// W pliku np. services/permitService.ts (zaadaptowane z quickstartu Circle)
import { maxUint256, erc20Abi as viemErc20Abi, parseErc6492Signature, getContract,
encodeFunctionData, PublicClient, WalletClient, Account } from 'viem';

```

```

// ABI dla nonces i version, jeśli nie ma ich w standardowym erc20Abi z viem
export const eip2612Abi = [
 ...viemErc20Abi,
 {
 inputs: [{ internalType: 'address', name: 'owner', type: 'address' }],
 name: 'nonces',
 outputs: [{ internalType: 'uint256', name: '', type: 'uint256' }],
 stateMutability: 'view',
 type: 'function',
 },
 {
 inputs: [],
 name: 'version', // Niektóre implementacje USDC mogą nie mieć 'version', inne tak
 outputs: [{ internalType: 'string', name: '', type: 'string' }],
 stateMutability: 'view',
 type: 'function',
 },
] as const; // Ważne dla type-safety z viem

export async function getPermitDataForPaymaster({
 publicClient,
 usdcContractAddress,
 ownerAddress, // Adres SCA lub EOA fana
 spenderAddress, // Adres Circle Paymaster
 value, // Kwota pozwolenia (np. 10 USDC w najmniejszych jednostkach)
 chainId,
}: { /* ... definicje typów ... */ }) {
 const tokenContract = getContract({
 address: usdcContractAddress,
 abi: eip2612Abi,
 client: publicClient,
 });

 let tokenName: string;
 let tokenVersion: string = "1"; // Domyślna wersja, jeśli kontrakt jej nie ma

 try {
 tokenName = await tokenContract.read.name();
 } catch (e) {
 tokenName = "USD Coin"; // Fallback
 console.warn("Could not fetch token name, using fallback.");
 }

 try {
 // Niektóre kontrakty USDC (szczególnie starsze lub na L2) mogą nie mieć funkcji
 version()
 const versionResult = await tokenContract.read.version();
 if (versionResult) tokenVersion = versionResult;
 }
}

```

```

 } catch (e) {
 console.warn("Token version() function not found or failed, using default '1'. This is
common for some USDC contracts.");
 }

```

```

const nonce = await tokenContract.read.nonces([ownerAddress]);

```

```

return {
 types: {
 Permit: [
 { name: 'owner', type: 'address' },
 { name: 'spender', type: 'address' },
 { name: 'value', type: 'uint256' },
 { name: 'nonce', type: 'uint256' },
 { name: 'deadline', type: 'uint256' },
],
 },
 primaryType: 'Permit',
 domain: {
 name: tokenName,
 version: tokenVersion,
 chainId: chainId,
 verifyingContract: usdcContractAddress,
 },
 message: {
 owner: ownerAddress,
 spender: spenderAddress,
 value,
 nonce,
 deadline: maxUint256, // Kluczowe dla Paymastera ERC-4337
 },
};
}

```

```

export async function signPermitForUsdcPaymaster({
 publicClient, // Do odczytu danych z kontraktu
 walletClient, // Do podpisania przez użytkownika
 account, // Konto fana (obiekt Account z viem)
 usdcContractAddress,
 paymasterAddress, // Spender
 permitAmount, // np. 10_000_000n dla 10 USDC (6 miejsc po przecinku)
}: { /* ... definicje typów ... */ }) {
 const permitTypedData = await getPermitDataForPaymaster({
 publicClient,
 usdcContractAddress,
 ownerAddress: account.address,
 spenderAddress: paymasterAddress,
 value: permitAmount,

```

```

 chainId: publicClient.chain.id,
 });

 const signature = await walletClient.signTypedData({
 account: account, // Konto, które podpisuje
 ...permitTypedData,
 });

 // Opcjonalna weryfikacja podpisu po stronie klienta (dobra praktyka)
 const isValid = await publicClient.verifyTypedData({
 address: account.address,
 signature: signature,
 ...permitTypedData,
 });

 if (!isValid) {
 throw new Error(`Nieprawidłowy podpis Permit dla ${account.address}: ${signature}`);
 }

 // parseErc6492Signature może być potrzebne, jeśli podpis jest opakowany (np. dla
 smart accounts)
 // Dla EOA, `signature` powinno być już w odpowiednim formacie.
 // const { signature: rawSignature } = parseErc6492Signature(signature);
 // return rawSignature;
 return signature;
}
...

```

### 3. **\*\*Przygotowanie Danych dla Kontraktu Paymastera (`paymasterAndData` lub `paymasterData`):\*\***

\* Po uzyskaniu podpisanego `permitSignature` od fana, frontend musi zakodować te dane w formacie oczekiwanym przez kontrakt Circle Paymaster i EntryPoint.

\* Format ten zazwyczaj obejmuje: typ operacji (dla Circle to zwykle `0`, oznaczający `MODE\_SPONSORED\_ERC20`), adres kontraktu USDC, kwotę pozwolenia (`permitAmount`) oraz sam `permitSignature`.

```

```typescript
// W logice wysyłania napiwku, po uzyskaniu permitSignature
import { encodePacked } from 'viem';

// Załóżmy, że mamy:
// const permitAmount = 10000000n; // 10 USDC (6 miejsc po przecinku)
// const fanAccount = eip7702SmartAccount; // Konto fana (EIP-7702 lub inne SCA)
// const permitSignature = await signPermitForUsdcPaymaster({ ... });

// Format dla danych Paymastera (może się różnić w zależności od wersji
EntryPoint/Paymastera)
// Dla Circle Paymaster v0.7/v0.8, dane zwykle zawierają:

```



```

// uint8 mode (0 dla sponsorowania z opłatą w tokenie)
// address feeTokenAddress (adres USDC)
// uint256 maxTokenAmount (kwota z permitu)
// bytes permit (podpisany permit)
const paymasterCallData = encodePacked(
  ['uint8', 'address', 'uint256', 'bytes'],
  [0, usdcContractAddress, permitAmount, permitSignature]
);

// Obiekt przekazywany do Bundlera/SmartAccountClient
const paymasterMiddlewareObject = {
  paymaster: paymasterAddress, // Adres kontraktu Circle Paymaster
  paymasterData: paymasterCallData,
  // Poniższe limity gazu są przykładami z dokumentacji Circle, mogą wymagać
dostosowania
  paymasterVerificationGasLimit: 200000n,
  paymasterPostOpGasLimit: 15000n,
  // isFinal: true, // Niektóre biblioteki mogą wymagać tego pola dla Circle Paymaster
};
...

```

****Faza 3: Konstrukcja i Wysłanie `UserOperation` przez Bundlera (Frontend TipJar)****

Po przygotowaniu danych dla Paymastera, następnym krokiem jest skonstruowanie i wysłanie `UserOperation` (UO) za pośrednictwem Bundlera. `UserOperation` to pseudo-transakcja w standardzie ERC-4337, która opisuje akcję do wykonania (np. wysłanie napiwku) oraz sposób pokrycia opłat za gaz (przez Paymastera).

1. ****Inicjalizacja Klienta Bundlera (`BundlerClient` z `viem`):****

* Frontend TipJar musi połączyć się z publicznie dostępnym serwisem Bundlera. Bundlerzy to podmioty w sieci ERC-4337, które odbierają `UserOperations`, pakują je w rzeczywiste transakcje blockchainowe i wysyłają do kontraktu EntryPoint.

* Przykładowi dostawcy Bundlerów to Pimlico, Alchemy, Biconomy, Stackup. Należy wybrać jednego i użyć jego RPC URL. Niektóre mogą wymagać klucza API.

```

```typescript
// W logice frontendu, gdzie wysyłany jest napiwek
import { createBundlerClient as createViemBundlerClient, ENTRYPOINT_ADDRESS_V07 }
from 'viem/account-abstract';
// Użyj adresu EntryPoint zgodnego z wersją Paymastera Circle, np. V07 dla Paymastera
v0.7/v0.8
// Dokumentacja Circle "Paymaster Addresses and Events" powinna precyzować
kompatybilny EntryPoint.
// Quickstart dla Paymaster v0.8 z EIP-7702 używa `toSimple7702SmartAccount`, który
jest kompatybilny z EntryPoint.

```

```

// const fanSmartAccount = eip7702SmartAccount; // Konto fana (EIP-7702 lub inne SCA)

```

```

// const currentChain = arbitrumSepolia; // Aktywna sieć
// const pimlicoApiKey = process.env.NEXT_PUBLIC_PIMLICO_BUNDLER_API_KEY;
// const pimlicoBundlerUrl =
`${process.env.NEXT_PUBLIC_PIMLICO_BUNDLER_URL_ARBITRUM_SEPOLIA}${pimlicoApiKey ? `?apikey=${pimlicoApiKey}` : ''}`;

```

```

const bundlerClient = createViemBundlerClient({
 chain: currentChain,
 transport: http(pimlicoBundlerUrl), // URL do RPC Bundlera
 entryPoint: entryPointAddress, // Adres kontraktu EntryPoint dla danej sieci i wersji
});

```

## 2. **\*\*Przygotowanie Danych Wywołania Kontraktu (`callData`) dla Napiwku:\*\***

\* Celem jest wysłanie napiwku, czyli wykonanie funkcji `transfer` na kontrakcie USDC. Frontend musi zakodować to wywołanie.

```

````typescript
import { encodeFunctionData } from 'viem';

// const creatorWalletAddress = '0x...'; // Adres portfela DCW twórcy
// const tipAmountInSmallestUnit = 1000000n; // Napiwek 1 USDC (6 miejsc po przecinku)

const tipCallData = encodeFunctionData({
  abi: eip2612Abi, // Lub standardowe erc20Abi, jeśli nie używamy niestandardowych
  functionName: 'transfer',
  args: [creatorWalletAddress, tipAmountInSmallestUnit],
});

```

3. ****Konstrukcja i Wysłanie `UserOperation`:****

* `UserOperation` to obiekt zawierający wszystkie niezbędne informacje. Dla transakcji sponsorowanej przez Paymastera, kluczowe jest dołączenie `paymasterAndData` (lub `paymaster`, `paymasterData` w zależności od wersji EntryPoint i używanej biblioteki klienckiej).

* Dla kont EOA działających jako SCA przez EIP-7702, może być wymagane dodatkowe podpisanie `authorization` przez właściciela EOA, jak pokazano w quickstarcie Circle dla Paymaster v0.8.

* Biblioteki takie jak `viem/account-abstraction` (lub `@alchemy/aa-core`, `permissionless`) dostarczają metod do estymacji gazu dla UO (`estimateUserOperationGas`), wypełnienia brakujących pól i wysłania UO (`sendUserOperation`).

```

````typescript
// Kontynuacja logiki frontendu
// const fanSmartAccount = eip7702SmartAccount;

```

```
// const paymasterMiddleware = paymasterMiddlewareObject; // Przygotowane wcześniej
// dane Paymastera
```

```
try {
 // Dla EIP-7702 (jak w quickstarcie Circle Paymaster v0.8):
 // Może być konieczne podpisanie autoryzacji przez oryginalne EOA (owner),
 // aby eip7702SmartAccount mogło działać w jego imieniu.
 // To zależy od konkretnej implementacji `toSimple7702SmartAccount` i `signer`
```

```
 // const nonceForAuth = await publicClient.getTransactionCount({ address:
 fanEOA_Account.address, blockTag: 'pending' });
 // const authorization = await fanEOA_Account.signAuthorization({ // Zakładając, że
 fanEOA_Account ma metodę signAuthorization
 // chainId: currentChain.id,
 // nonce: nonceForAuth,
 // contractAddress: fanSmartAccount.authorization.address, // Adres z
 `toSimple7702SmartAccount`
 // });
 // Uwaga: `signAuthorization` to specyficzna metoda, może wymagać dostosowania do
 używanego `signer`
```

```
 // Wysyłanie UserOperation
 const userOpHash = await bundlerClient.sendUserOperation({
 account: fanSmartAccount, // Konto, które wykonuje operację (EIP-7702 lub SCA)
 userOperation: {
 callData: tipCallData, // Zakodowane wywołanie transferu USDC
 // Wiele bibliotek automatycznie wypełni `sender`, `nonce`, `initCode` (jeśli potrzebny)
 // oraz oszacuje `callGasLimit`, `verificationGasLimit`, `preVerificationGas`
 // `maxFeePerGas` i `maxPriorityFeePerGas` zostaną pobrane z Bundlera lub
 estymatora
 }
 });
```

```
 // Przekazanie danych Paymastera - sposób zależy od biblioteki i wersji EntryPoint
 // Dla viem i EntryPoint v0.7/v0.8, można użyć middleware lub przekazać jako obiekt:
 paymaster: paymasterMiddleware.paymaster,
 paymasterData: paymasterMiddleware.paymasterData,
 paymasterVerificationGasLimit: paymasterMiddleware.paymasterVerificationGasLimit,
 paymasterPostOpGasLimit: paymasterMiddleware.paymasterPostOpGasLimit,
 },
 // Dla EIP-7702 i niektórych implementacji, może być potrzebne:
 // authorization: authorization,
 });
```

```
console.log('UserOperation hash:', userOpHash);
// Poinformuj fana, że transakcja jest przetwarzana
```

```
// Oczekiwanie na potwierdzenie (opcjonalne, ale dobre dla UX)
const receipt = await bundlerClient.waitForUserOperationReceipt({ hash: userOpHash });
console.log('UserOperation Receipt:', receipt);
```

```

console.log('Transaction hash:', receipt.receipt.transactionHash);
// Poinformuj fana o sukcesie i wyświetl hash transakcji

} catch (error) {
 console.error('Błąd podczas wysyłania UserOperation:', error);
 // Poinformuj fana o błędzie
}
...

```

**\*\*\*Ważne:\*\*** Sposób przekazywania danych ``paymaster`` do ``sendUserOperation`` może się różnić w zależności od użytej biblioteki (``viem``, ``@alchemy/aa-sdk``, ``permissionless``) i wersji standardu EntryPoint. Należy zawsze konsultować się z aktualną dokumentacją biblioteki i przykładami Circle. Niektóre biblioteki oferują "middleware" dla Paymastera, który automatycznie dołącza odpowiednie dane.

---

#### **\*\*Faza 4: Monitorowanie i Potwierdzenie Transakcji (Frontend i Opcjonalnie Backend TipJar)\*\***

Po wysłaniu ``UserOperation`` do Bundlera, ważne jest, aby poinformować fana o statusie operacji i ostatecznie potwierdzić jej wykonanie.

##### 1. **\*\*Potwierdzenie po Stronie Frontendu:\*\***

\* Jak pokazano wyżej, po otrzymaniu ``userOpHash`` od metody ``sendUserOperation``, frontend może użyć metody ``bundlerClient.waitForUserOperationReceipt({ hash: userOpHash })``. Ta metoda będzie okresowo odpytywać Bundlera o status ``UserOperation``, aż zostanie ona włączona do bloku i otrzyma potwierdzenie (lub zakończy się błędem).

\* Po otrzymaniu ``receipt`` (paragonu), frontend ma dostęp do ``receipt.receipt.transactionHash`` (hash rzeczywistej transakcji na blockchainie) oraz ``receipt.success`` (boolean wskazujący, czy operacja się powiodła).

**\*\*UX:\*\*** W tym czasie frontend powinien wyświetlać stan ładowania (np. "Przetwarzanie napiwku..."). Po otrzymaniu paragonu:

\* Jeśli ``receipt.success === true``: Wyświetl komunikat o powodzeniu (np. "Napiwek wysłany pomyślnie! Dziękujemy!"), ewentualnie z linkiem do explorera bloków z ``transactionHash``.

\* Jeśli ``receipt.success === false``: Wyświetl komunikat o błędzie, ewentualnie z informacjami z paragonu, jeśli są pomocne.

##### 2. **\*\*Monitorowanie i Weryfikacja po Stronie Backendu (Opcjonalne, ale Zalecane dla Zwiększenia Niezawodności):\*\***

\* Frontend, po uzyskaniu ``transactionHash`` (lub ``userOpHash``), może wysłać tę informację do backendu TipJar.

\* Backend TipJar może:

**\*\*Nasłuchiwać na Zdarzenia (Events) Kontraktu Circle Paymaster:\*\*** Dokumentacja Circle ("Paymaster Addresses and Events") opisuje zdarzenie ``UserOperationSponsored``, które jest emitowane przez kontrakt Paymastera po pomyślnym przetworzeniu i opłaceniu ``UserOperation``. Backend mógłby subskrybować te zdarzenia (używając klienta RPC dla

danej sieci i adresu Paymastera), aby otrzymywać potwierdzenia sponsorowanych transakcji i logować je wewnętrznie. To dostarcza niezależnego potwierdzenia.

- \* Atrybuty zdarzenia ``UserOperationSponsored`` to m.in. ``token`` (adres USDC), ``sender`` (adres SCA/EOA fana), ``userOpHash``, ``actualTokenNeeded`` (finalny koszt w USDC dla fana).

- \* **Monitorować Saldo Portfela Twórcy:** Niezależnie od powyższego, backend powinien monitorować saldo portfela DCW twórcy, aby potwierdzić otrzymanie napiwku USDC. Można to robić przez odpytywanie API Circle dla portfeli DCW lub przez nasłuchiwanie na transfery ERC20 na adres portfela twórcy.

- \* **Korzyści z Weryfikacji Backendowej:** Dodatkowa warstwa pewności, możliwość logowania wszystkich transakcji w systemie TipJar, ułatwienie rozwiązywania ewentualnych sporów lub problemów.

---

## **\*\*Podsumowanie Kluczowych Wyzwań Integracyjnych i Najlepszych Praktyk:\*\***

- \* **Zarządzanie Złożonością ERC-4337:** Standard Account Abstraction jest elastyczny, ale implementacja jego komponentów (``UserOperations``, ``Bundlers``, ``Paymasters``, różne wersje ``EntryPoint``) wymaga staranności. Wybór odpowiednich, dobrze wspieranych bibliotek klienckich (``viem/account-abstraction``, ``@alchemy/aa-sdk``) jest kluczowy.

- \* **Wybór i Konfiguracja Bundlera:** Należy wybrać niezawodnego dostawcę Bundlera i odpowiednio skonfigurować z nim komunikację (w tym ewentualne klucze API).

- \* **Zgodność Wersji EntryPoint:** Upewnij się, że używana wersja kontraktu ``EntryPoint`` (np. v0.6, v0.7, v0.8) jest spójna i wspierana przez portfel fana (lub jego reprezentację SCA), Bundlera oraz Circle Paymastera na danej sieci.

- \* **Implementacja i Podpisywanie `Permit` (EIP-2612):** Prawidłowe wygenerowanie danych ``Permit``, ich podpisanie przez portfel fana i zakodowanie dla Paymastera jest krytyczne. Należy dokładnie przetestować ten przepływ.

- \* **Doświadczenie Użytkownika (UX):** Cały proces, mimo swojej złożoności "pod maską", musi być dla fana jak najprostszy. Jasne komunikaty o tym, co się dzieje (np. "Podpisz pozwolenie na użycie USDC na opłaty", "Oczekiwanie na potwierdzenie transakcji"), wskaźniki postępu i obsługa błędów są niezbędne.

- \* **Testowanie na Testnetach:** Zanim integracja trafi na mainnet, musi być gruntownie przetestowana na odpowiednich sieciach testowych (np. Arbitrum Sepolia, Polygon Amoy, Ethereum Sepolia), korzystając z tokenów USDC z Circle Faucet.

- \* **Aktualizacje i Zmiany w Standardach:** Ekosystem Account Abstraction wciąż ewoluuje. Należy być przygotowanym na aktualizacje bibliotek, kontraktów ``EntryPoint`` i potencjalne zmiany w API Bundlerów czy Paymasterów.

## **\*\*Rekomendacja Końcowa dla TipJar:\*\***

Integracja z Circle Paymaster to znaczący krok w kierunku ułatwienia fanom korzystania z TipJar, szczególnie tym, którzy preferują własne portfele EOA. Ze względu na złożoność, **zaleca się rozpoczęcie implementacji dla jednej lub dwóch kluczowych, dobrze wspieranych sieci L2 (np. Polygon PoS, Arbitrum One)**, na których dostępne są zarówno Circle Paymaster v0.8, jak i niezawodne publiczne Bundlery.

Dokładne śledzenie dokumentacji Circle, wybranej biblioteki do obsługi Account Abstraction (np. `viem`) oraz dokumentacji używanego Bundlera będzie niezbędne do pomyślnej integracji. Wykorzystanie okresu promocyjnego Circle (zniesiona opłata 10% do 1 lipca 2025) może być dodatkowym motywatorem do wczesnego wdrożenia tej funkcjonalności.